A D PATEL INSTITUTE OF TECHNOLOGY
(The Charutar Vidya Mandal University)
GSIRE An ISO 9001:2015 Certified Institute

Smart Delivery Route Planning System

A Comparative Analysis of Greedy, Brute Force and Dynamic Programming Algorithms

A Design and Analysis of Algorithms Mini Project report submitted
In partial fulfilment of the requirements for the Degree of
Bachelor of Technology
In
Information Technology
Semester – V

By
Binsi Patel (12402080601020)

Faculty Guide
Minakshi Patil
Batch 1A8

Academic Year 2026-2027 (Odd)

Department of Information Technology
A D Patel Institute of Technology
New Vallabh Vidyanagar, Vitthal Udyognagar – 388121
Anand (Gujarat), India

CERTIFICATE

This is to certify that the Design and Analysis of Algorithms Mini Project work embodied in this report entitled, "Smart Delivery Route Planning System: A Comparative Analysis of Greedy, Brute Force and Dynamic Programming Algorithms" was carried out by Binsi Patel (12402080601020) at A D Patel Institute of Technology for partial fulfilment of B.Tech. degree to be awarded by the Charutar Vidya Mandal (CVM) University. This project report work has been carried out under my supervision and is to the satisfaction of the department.

Date:

Place:

Guide **Minakshi Patil** Faculty Guide	**Head of Department** **Priyang Bhatt** Professor & Head

Acknowledgement

I would like to express my sincere thanks to my faculty guide, Minakshi Patil, for guiding me throughout the preparation of this mini project on the Smart Delivery Route Planning System. The support, advice and encouragement received during the work helped me understand the topic in depth and complete it successfully.

I am thankful to the Department of Information Technology and A D Patel Institute of Technology for providing the opportunity, the laboratory facilities and the learning resources needed to build and present this project in the course Design and Analysis of Algorithms.

I would like to thank my parents for their constant support and motivation. Their encouragement helped me stay focused and complete this work on time.

I am also grateful to my friends and classmates who shared ideas, tested the application and supported me whenever needed.

Lastly, I thank everyone who directly or indirectly helped me in completing this report. This project has been a valuable learning experience for me.

Binsi Patel

Abstract

Delivery companies, courier services and food-delivery operators all face the same planning question: in what order should a vehicle visit its delivery locations so that the total distance travelled is as small as possible? When the vehicle must start at a warehouse, visit every location exactly once and return, this question is the classical Travelling Salesman Problem (TSP), a well-known problem in the study of algorithm design.

This mini project presents the design, implementation and analysis of a Smart Delivery Route Planning System that solves the problem with three different algorithms and compares them on the same input. The first is the Nearest Neighbour algorithm, a greedy heuristic that is fast but not guaranteed to be optimal. The second is Brute Force, which examines every possible visiting order and is always optimal but becomes impractical very quickly. The third is the Held–Karp Dynamic Programming algorithm, which stores and reuses the results of subproblems using a bitmask representation of visited locations and also gives an optimal route.

The system is written in pure Python with no external packages. The algorithms are placed in the file route_algorithms.py, and a Tkinter graphical interface in smart_delivery_gui.py accepts the locations and distances, validates them, mirrors the reverse distances, shows the route, total distance and execution time of every algorithm, and draws a performance graph. On the sample input of five locations, Nearest Neighbour produced a route of 91 units while both Brute Force and Dynamic Programming found the optimal route of 82 units. In the timing experiment, Dynamic Programming overtook Brute Force from seven locations, and at nine locations Brute Force took 37.63 ms compared with 1.73 ms for Dynamic Programming, about 22 times longer. The results agree with the theoretical complexities of $O(n^2)$, $O(n!)$ and $O(n^2 \cdot 2^n)$, and show that the best algorithm depends on the balance between running time, memory and the need for an optimal route.

Table of Contents

Certificate ..... **I**
Acknowledgement ..... **II**
Abstract ..... **III**
Table of Contents ..... **IV**
CHAPTER – 1 INTRODUCTION **1**
1.1 Background 1
1.2 The Delivery Route Problem 1
1.3 Problem Statement 1
1.4 Objectives 2
1.5 Scope of the Project 2
CHAPTER – 2 BACKGROUND AND TECHNOLOGY OVERVIEW **3**
2.1 Existing Approaches 3
2.2 Python and the Standard Library 3
2.3 Tkinter 3
CHAPTER – 3 METHODOLOGY **5**
3.1 Development Approach 5
3.2 Requirements 6
3.3 Design Choices 6
CHAPTER – 4 ALGORITHM DESIGN AND ANALYSIS **7**
4.1 Distance Matrix 7
4.2 Nearest Neighbour Algorithm 7
4.3 Brute Force Algorithm 7
4.4 Held–Karp Dynamic Programming 8
4.5 Complexity Analysis 9
4.6 Worked Example 9
CHAPTER – 5 SYSTEM DESIGN **11**
5.1 Architecture 11
5.2 Working Flow 11
5.3 Folder Structure 12
5.4 Functions and Data Structures 12
CHAPTER – 6 IMPLEMENTATION **13**
6.1 Route Distance Function 13
6.2 Nearest Neighbour Code 13
6.3 Brute Force Code 13
6.4 Held–Karp Code 14
6.5 Graphical User Interface 14
CHAPTER – 7 RESULTS AND TESTING **16**
7.1 Output Screens 16
7.2 Route Comparison 17
7.3 Performance Measurement 17
7.4 Test Cases 18
7.5 Discussion of Results 19
CHAPTER – 8 DISCUSSION **21**
8.1 Advantages 21
8.2 Limitations 21
8.3 Future Scope 21
CHAPTER – 9 CONCLUSION **22**
REFERENCES **23**
APPENDIX A – SOURCE CODE **24**

CHAPTER – 1 INTRODUCTION

1.1 Background

Every day, delivery vehicles carry parcels, groceries, medicines and meals from a central point to many different addresses. The order in which the addresses are visited decides how many kilometres the vehicle travels, how much fuel it burns and how long each customer waits. A poor order wastes money and time, while a good order saves both. As e-commerce and food delivery have grown, even a small saving per trip becomes large when multiplied over thousands of trips.

Choosing a good order is a problem of algorithm design. A person can often guess a reasonable order for three or four stops, but as the number of stops grows, the number of possible orders grows explosively. Computer algorithms are therefore needed, and different algorithms offer different trade-offs between the speed of finding an answer and the quality of that answer. This mini project studies those trade-offs through a working implementation.

1.2 The Delivery Route Problem

A delivery vehicle starts at a warehouse, must visit every delivery location exactly once, and must return to the warehouse. The aim is to make the total distance of this tour as small as possible. In computer science this is the **Travelling Salesman Problem (TSP)**. It is a standard example in the study of algorithms because it is easy to state, appears in many real situations, and has no known algorithm that solves every instance quickly. The decision version of the problem is NP-complete.

In this project the locations and the distances between them are modelled with a **distance matrix**. If there are n locations including the warehouse, the matrix has n rows and n columns, and the entry in row i and column j is the distance from location i to location j. The warehouse is always location 0. The system uses symmetric distances, so the distance from i to j equals the distance from j to i, and the distance from a location to itself is zero.

1.3 Problem Statement

Choosing a delivery order by hand does not compare all possible routes, and a quick rule such as always going to the nearest place does not guarantee the shortest complete route. Exact methods guarantee the best route but may take far too long as the number of locations increases. There is a need for a simple tool that accepts a set of locations and distances, finds the delivery route with several different algorithms, and shows clearly how their routes, total distances and running times differ.

Item	Description
Input	Set of delivery locations; distance matrix between locations; warehouse as the fixed start
Output	Complete delivery route; total distance travelled; execution time of each algorithm
Objective	Minimise the total distance of the complete route, including the return trip
Constraints	Each location is visited exactly once; the route starts and ends at the warehouse; distances are valid and non-negative; distances are symmetric

Table 1. Formal definition of the problem.

1.4 Objectives

- To model the delivery locations and their distances with a distance matrix.
- To implement three algorithms: Nearest Neighbour, Brute Force and Held–Karp Dynamic Programming.
- To calculate the visiting order and the total distance for each algorithm.
- To measure and compare the execution time for different input sizes.
- To analyse the time and space complexity of each algorithm.
- To present the route comparison and the measurements through a simple graphical interface.

1.5 Scope of the Project

The project covers the implementation and comparison of three route-planning algorithms for one warehouse and between 2 and 10 locations, a Tkinter interface for input, validation and visualisation, and a timing experiment for 4 to 10 locations. It is a demonstration of the route-planning concept. It is not connected to live maps, traffic information or any real company's data. Real road distances, one-way streets, traffic, multiple vehicles and delivery time windows are outside the present scope and are listed as future scope in Chapter 8.

This report is organised as follows. Chapter 2 gives the background and technology overview. Chapter 3 explains the methodology. Chapter 4 describes the algorithms and their complexity. Chapter 5 presents the system design. Chapter 6 explains the implementation. Chapter 7 presents the results and testing. Chapter 8 discusses advantages, limitations and future scope, and Chapter 9 concludes the report.

CHAPTER – 2 BACKGROUND AND TECHNOLOGY OVERVIEW

2.1 Existing Approaches

Several approaches can be used to decide a delivery order. They differ in how much effort they need and how good the resulting route is. Table 2 summarises the four approaches considered in this project.

Approach	Working principle	Limitation
Manual route selection	A person chooses the visiting order	May not compare all possible routes
Greedy selection	Selects the nearest unvisited location	Does not guarantee the shortest complete route
Exhaustive search	Evaluates every possible visiting order	Computationally expensive
Dynamic Programming	Stores and reuses subproblem results	Exponential time and memory for this problem

Table 2. Existing approaches to route selection.

A comparative implementation is useful because it shows the trade-offs between computational efficiency, route optimality and memory usage side by side. Nearest Neighbour is a fast heuristic. Brute Force is optimal but costly. Dynamic Programming is also optimal and, for larger inputs, much cheaper than Brute Force, although its practical position depends on the input size and the implementation.

2.2 Python and the Standard Library

The whole system is written in Python. Python was chosen because its syntax is short and readable, which keeps the algorithms close to their pseudocode, and because its standard library already provides everything the project needs. No external package has to be installed. The modules used are listed below.

- `itertools`: `permutations` generates every visiting order for Brute Force, and `combinations` generates the subsets of locations for Dynamic Programming.
- `time`: `time.perf_counter()` gives a high-resolution clock for measuring the execution time of each algorithm in milliseconds.
- `random`: generates symmetric distance matrices for the performance test. A fixed seed makes the matrix for each size repeatable.
- `tkinter`: provides the graphical user interface, including the tabs, the table of entry boxes, the results table and the drawing canvas.

2.3 Tkinter

Tkinter is the standard graphical toolkit that is bundled with Python. The interface in this project uses the themed widget set `ttk`. A `Notebook` widget creates the three tabs. `Entry` widgets collect the distances, a `Treeview` widget shows the result table, a `Combobox` selects the maximum number of locations for the performance test, and a `Canvas` widget is used to draw the performance graph from basic lines, ovals and text. Using the canvas means that the graph needs no plotting library such as Matplotlib.

CHAPTER – 3 METHODOLOGY

3.1 Development Approach

The project was developed in stages. The problem was first defined formally, and the distance matrix was chosen as the common input format. The three algorithms were then written and tested in a separate file, route_algorithms.py, so that they could be checked without any interface. After the algorithms returned correct results on the sample input, the Tkinter interface was built around them, and the timing experiment and graph were added last. The steps are shown in Figure 1.

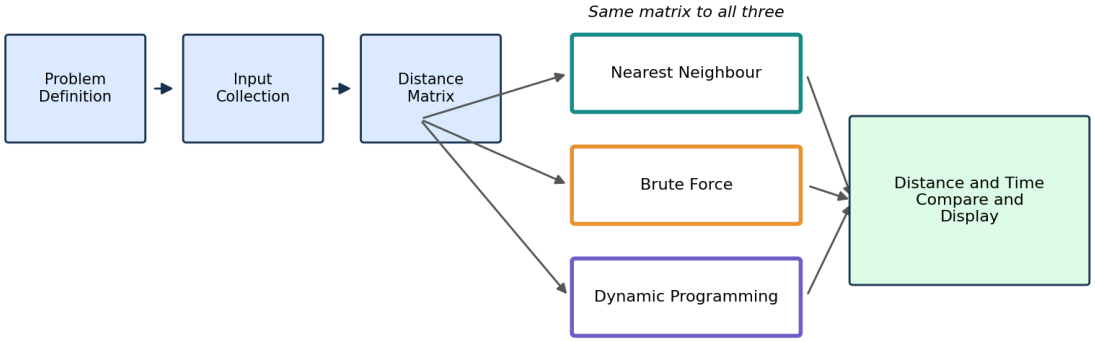


Figure 1. Proposed solution and workflow.

1. **Problem definition:** a vehicle starts at the warehouse, visits every location once and returns, minimising distance.
2. **Input collection:** the user enters the location names and the distances.
3. **Distance matrix:** the system validates the input and stores a symmetric matrix.
4. **Algorithm execution:** the same matrix is given to Nearest Neighbour, Brute Force and Held–Karp, so the comparison is fair.
5. **Distance and time:** the total distance and the run time of each route are recorded.
6. **Compare and display:** the results are shown in the results table and in the performance graph.

3.2 Requirements

Type	Requirement
Hardware	Laptop or desktop with at least 4 GB RAM
Operating system	Windows, Linux or macOS
Language	Python 3.x (Tkinter must be included, which is the default in most installers)
Packages	None. Only the Python standard library is used
Display	A screen able to show an 900 × 620 pixel window
Editor	Any code editor, such as VS Code or IDLE

Table 3. Hardware and software requirements.

3.3 Design Choices

Three algorithms were chosen because each represents a different design technique. Nearest Neighbour represents the greedy technique and acts as a fast heuristic. Brute Force represents exhaustive search and provides an optimal baseline for small inputs. Held–Karp represents dynamic programming, which gets optimisation through reusable subproblems. Comparing the three on one matrix shows clearly how the choice of technique changes both the speed and the quality of the answer.

The warehouse is fixed as location 0 and is always the starting point. This removes equivalent rotations of the same tour and reduces the number of orders that Brute Force has to check from n! to (n − 1)!. Distances are assumed to be symmetric, which lets the user fill in only the upper triangle of the table. The input size is limited to 10 locations because Brute Force and Dynamic Programming both grow exponentially, and larger inputs would make the program appear to hang. The interface uses only standard-library modules so that the project runs on any computer with Python and nothing else.

CHAPTER – 4 ALGORITHM DESIGN AND ANALYSIS

In this chapter, n is the total number of locations including the warehouse, and `dist[i][j]` is the distance from location i to location j. The warehouse is location 0. A route is a list of locations that begins with 0 and contains every location exactly once. The total distance of a route includes the return trip from the last location to the warehouse.

4.1 Distance Matrix

The distance matrix is a two-dimensional list. Because distances are symmetric, `dist[i][j]` equals `dist[j][i]`, and the interface asks only for the entries above the diagonal and copies each value to its mirror position. The diagonal entries are zero. The function `calculate_route_distance(route, dist)` adds the distance of each consecutive pair of locations in the route and then adds the distance from the last location back to the warehouse.

4.2 Nearest Neighbour Algorithm

Nearest Neighbour is a **greedy** algorithm. It starts at the warehouse and repeatedly moves to the closest location that has not yet been visited, until all locations are visited. Finally it returns to the warehouse. The data structures are the distance matrix, the route list and a visited list. The greedy choice is made locally at each step, so the algorithm never reconsiders an earlier decision. This makes it fast, but a short step now can force a long step later, so the final route is not guaranteed to be the shortest.

```
route = [warehouse]
mark warehouse visited
while unvisited locations remain:
    next = nearest unvisited location from current
    route.add(next)
    mark next visited
    current = next
distance = route distance + distance back to warehouse
return route
```

Complexity: for each of the n − 1 steps, the algorithm scans up to n locations to find the nearest unvisited one, giving time $O(n^2)$. It needs only the route and the visited list, so the extra space is O(n).

4.3 Brute Force Algorithm

Brute Force, also called exhaustive search, generates every possible visiting order, calculates the total distance of each, and keeps the smallest. With the warehouse fixed as the start, the remaining n − 1 locations can be arranged in (n − 1)! different orders. Because it examines

every possibility, it is guaranteed to find an optimal route. It uses `itertools.permutations` to generate the orders one by one, so it does not need to store them all.

```
fix warehouse as start
best = infinity
for each order of the other locations:
    d = route distance (including return trip)
    if d < best:
        best = d
        save route
return best route
```

Complexity: there are (n − 1)! orders, and each one takes O(n) time to total, so the time is O(n!) in every case. Only the current permutation and the best route are kept, so the space is O(n).

4.4 Held–Karp Dynamic Programming

The Held–Karp algorithm solves the problem with **dynamic programming**. The key observation is that the best route to visit a set of locations and end at a particular location can be built from best routes over smaller sets. Instead of recalculating these smaller routes again and again, as Brute Force does, Held–Karp calculates each one once and stores it.

A subset of the non-warehouse locations is represented as a **bitmask**: bit (k − 1) of the mask is 1 when location k is in the subset. The state `dp[(mask, j)]` stores the length of the shortest path that starts at the warehouse, visits exactly the locations in `mask`, and ends at location j. The recurrence is:

```
dp[(mask, j)] = min over k in mask, k != j of
                    dp[(mask without j, k)] + dist[k][j]

base case:  dp[({j}, j)] = dist[0][j]
answer   :  min over j of dp[(all locations, j)] + dist[j][0]
```

A second dictionary, `parent`, records which location k gave the minimum for each state. After the final answer is found, the route is rebuilt by starting at the best last location and following the parent entries back to the warehouse, and then reversing the list. The data structures are a dictionary of DP states, the parent map, and the bitmask.

Complexity: there are about $n \cdot 2^n$ states, and each state looks at up to n previous locations, so the time is $O(n^2 \cdot 2^n)$. The dp and parent tables hold one entry per state, so the space is $O(n \cdot 2^n)$. Since 2^n grows far more slowly than n!, Held–Karp scales to larger inputs than Brute Force, but it is still exponential in both time and memory.

4.5 Complexity Analysis

Algorithm	Best case	Average case	Worst case	Space	Optimal?
Nearest Neighbour	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(n)	Not guaranteed
Brute Force	O(n!)	O(n!)	O(n!)	O(n)	Yes
Held–Karp DP	$O(n^2 \cdot 2^n)$	$O(n^2 \cdot 2^n)$	$O(n^2 \cdot 2^n)$	$O(n \cdot 2^n)$	Yes

Table 4. Theoretical complexity (n = total locations including the warehouse; space excludes the input matrix).

The bounds are the same for the best, average and worst cases because the main loops or state enumeration always run in full, whatever the distances are. Nearest Neighbour always makes n − 1 selections that each scan the locations. Brute Force always evaluates all (n − 1)! orders, as it has no way to stop early. Held–Karp always fills every state of the table. Theory predicts how the cost grows with n, while the experiments in Chapter 7 measure the real time on one computer; the two should agree in the growth pattern even if the actual milliseconds depend on the machine.

4.6 Worked Example

The sample input has five locations: Warehouse (0), College (1), Hospital (2), Market (3) and Station (4). Table 5 shows its symmetric distance matrix.

	Warehouse	College	Hospital	Market	Station
Warehouse	0	10	15	20	13
College	10	0	20	25	17
Hospital	15	20	0	30	14
Market	20	25	30	0	18
Station	13	17	14	18	0

Table 5. Distance matrix of the sample input.

Nearest Neighbour starts at the Warehouse. The nearest location is College (10). From College, the unvisited locations are Hospital (20), Market (25) and Station (17), so it goes to Station. From Station it chooses Hospital (14) over Market (18). The only location left is Market, which is 30 from Hospital, and the return to the Warehouse costs 20. The route is Warehouse → College → Station → Hospital → Market → Warehouse, with total distance 10 + 17 + 14 + 30 + 20 = **91**.

Brute Force checks all (5 − 1)! = 24 orders. **Held–Karp** reaches the same answer by reusing subpath results. Both return the route Warehouse → College → Hospital → Station → Market

→ Warehouse, with total distance 10 + 20 + 14 + 18 + 20 = **82**. The greedy route is therefore 9 units longer than the optimum, because the cheap step from College to Station forces the expensive step from Hospital to Market later. Figure 2 shows both routes.

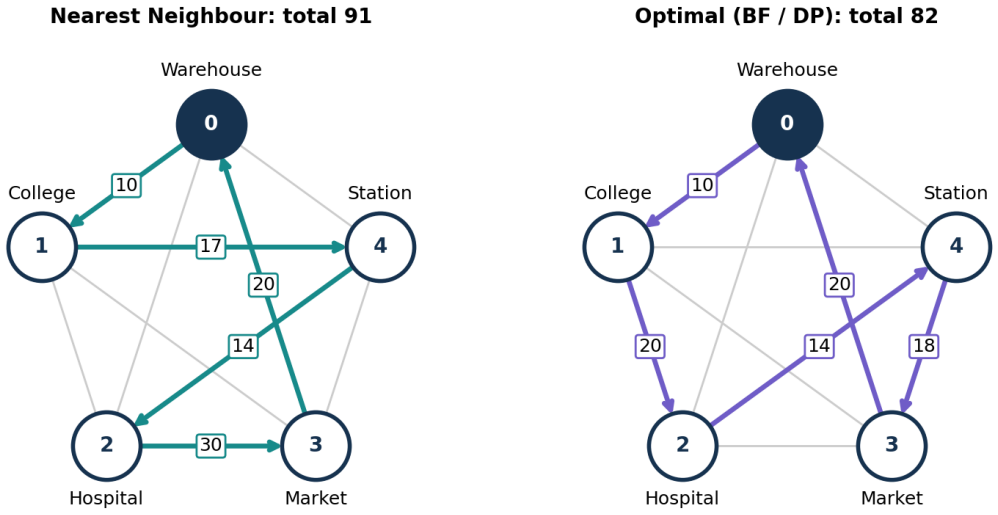


Figure 2. Route found by Nearest Neighbour (91) and the optimal route found by Brute Force and Dynamic Programming (82).

CHAPTER – 5 SYSTEM DESIGN

5.1 Architecture

The application is divided into four parts, as shown in Figure 3. The **User Interface** accepts the location names and the distances and displays the results. **Input Processing** validates the input, builds the distance matrix and mirrors the reverse distances. The **Algorithm Module** contains the three route-planning algorithms. **Results and Visualisation** calculates the total distance of each route, measures the execution time, compares the results and draws the performance graph. The algorithms are kept in a separate file from the interface, so they can be reused or tested without the GUI.

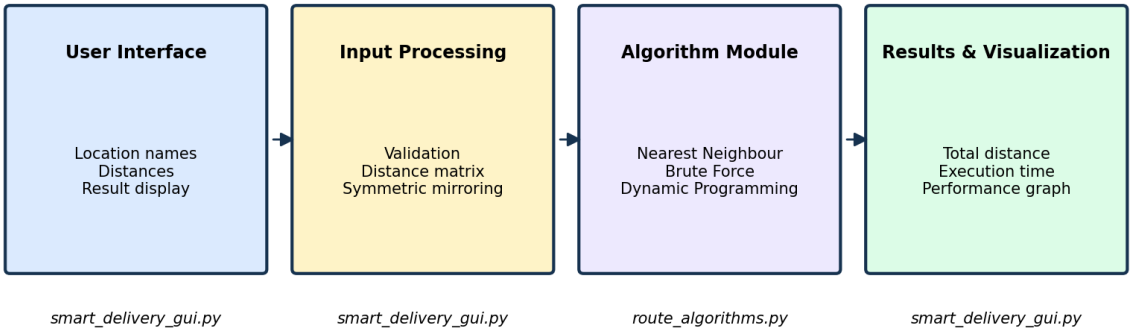


Figure 3. Architecture of the Smart Delivery Route Planning System.

5.2 Working Flow

1. The user runs `python smart_delivery_gui.py`, and the window opens on the first tab.
2. The user enters 2 to 10 different location names, with the warehouse first, and presses **Create distance table**. The system checks the number and uniqueness of the names.
3. A table of entry boxes appears. The user fills in only the upper triangle; the lower triangle shows “Auto-filled”. The **Load example** button fills the sample data.
4. The user presses **Calculate & compare routes**. The system reads every entry, rejects invalid or negative values and builds the symmetric matrix.
5. The same matrix is passed to Nearest Neighbour, Brute Force and Dynamic Programming. Each call is timed with `perf_counter`.
6. The second tab shows each algorithm’s route, total distance and time.

7. On the third tab, the user selects the maximum number of locations (4 to 10) and runs the performance test. The program times each algorithm on generated matrices, draws the graph and prints a table of the averages.

5.3 Folder Structure

```
smart_delivery/
  route_algorithms.py     Distance function and 3 algorithms
  smart_delivery_gui.py   Tkinter GUI, validation, benchmark, graph
```

5.4 Functions and Data Structures

Function / class	Purpose
calculate_route_distance()	Total route distance, including the return to the warehouse
nearest_neighbour()	Implements the greedy algorithm
brute_force()	Evaluates all visiting orders
dynamic_programming()	Implements Held–Karp dynamic programming
timed()	Runs an algorithm and returns its route, distance and time in ms
App.create_table()	Validates the names and builds the table of entry boxes
App.get_matrix()	Reads and validates the distances and builds the symmetric matrix
App.calculate()	Runs all three algorithms and fills the result table
App.benchmark()	Runs the timing experiment for 4 to n locations
App.draw_graph()	Draws the performance graph on the canvas

Table 6. Main functions of the program.

Data structure	Used for
Two-dimensional list	The distance matrix
List	The route and the visited locations
Dictionary	The DP states and the parent information
Bitmask (integer)	Represents a subset of locations in Held–Karp

Table 7. Data structures used.

CHAPTER – 6 IMPLEMENTATION

This chapter explains the important parts of the code. The complete source code of both files is given in Appendix A.

6.1 Route Distance Function

`calculate_route_distance` is used by both Nearest Neighbour and Brute Force. It sums the distance between every pair of consecutive locations and then adds the closing edge from the last location back to the first, which is the warehouse.

```
def calculate_route_distance(route, dist):
    total = 0
    for i in range(len(route) - 1):
        total += dist[route[i]][route[i + 1]]
    total += dist[route[-1]][route[0]]  # return trip
    return total
```

6.2 Nearest Neighbour Code

The list `visited` records which locations are used. The inner loop looks at every unvisited location and keeps the one with the smallest distance from the current location. The chosen location is appended to the route and becomes the new current location.

```
def nearest_neighbour(dist):
    n = len(dist)
    route = [0]
    visited = [False] * n
    visited[0] = True
    current = 0
    while len(route) < n:
        nxt = None
        for j in range(n):
            if not visited[j] and (nxt is None or
                                   dist[current][j] < dist[current][nxt]):
                nxt = j
        route.append(nxt)
        visited[nxt] = True
        current = nxt
    return route, calculate_route_distance(route, dist)
```

6.3 Brute Force Code

`itertools.permutations(range(1, n))` produces each ordering of the non-warehouse locations. The warehouse is prefixed to make a full route, and the route is kept if it is shorter than the best found so far.

```
def brute_force(dist):
    n = len(dist)
    best_route, best = None, INF
    for perm in itertools.permutations(range(1, n)):
        route = [0] + list(perm)
        d = calculate_route_distance(route, dist)
        if d < best:
            best, best_route = d, route
    return best_route, best
```

6.4 Held–Karp Code

The base states give the cost of going directly from the warehouse to a single location. The loops then process subsets in increasing size. For each subset and each possible end location j, the code removes j from the mask and tries every other location k in the subset as the previous stop, keeping the cheapest option and its parent. After the table is complete, the best final location is chosen by adding the return distance, and the route is rebuilt by walking the parent entries.

```
for size in range(2, n):
    for subset in combinations(range(1, n), size):
        mask = sum(1 << (k - 1) for k in subset)
        for j in subset:
            prev = mask ^ (1 << (j - 1))
            best, best_k = INF, None
            for k in subset:
                if k == j:
                    continue
                cost = dp[(prev, k)] + dist[k][j]
                if cost < best:
                    best, best_k = cost, k
            dp[(mask, j)] = best
            parent[(mask, j)] = best_k

full = (1 << (n - 1)) - 1
best, last = INF, None
for j in range(1, n):
    cost = dp[(full, j)] + dist[j][0]
    if cost < best:
        best, last = cost, j
```

6.5 Graphical User Interface

The interface is a single `App` class with a `ttk.Notebook` that holds three tabs.

- **Tab 1, Locations & Distances:** takes the comma-separated location names and builds a table of entry boxes. Only the upper triangle is editable. The method `get_matrix` converts every entry to a number, rejects invalid or negative values with a message box, and writes each value into both `m[i][j]` and `m[j][i]`. A **Load example** button fills in the sample data.

- **Tab 2, Route Comparison:** a `Treeview` table with one row per algorithm showing the route (as location names, including the return to the warehouse), the total distance and the time in milliseconds.
- **Tab 3, Performance Graph:** the user chooses a maximum size from 4 to 10 and runs the test. For each size, a random symmetric matrix with distances from 5 to 50 is generated using a fixed seed, so every algorithm sees the same input. Each algorithm is run three times and the average time is stored. `draw_graph` then draws the axes, grid lines, labels, legend and one coloured line per algorithm on a canvas, and a text table of the averages is printed below it.

The `timed` helper wraps every algorithm call with `time.perf_counter()` and converts the elapsed time to milliseconds.

CHAPTER – 7 RESULTS AND TESTING

7.1 Output Screens

The program is run with `python smart_delivery_gui.py`. Figures 4 to 6 show the three tabs of the running application.

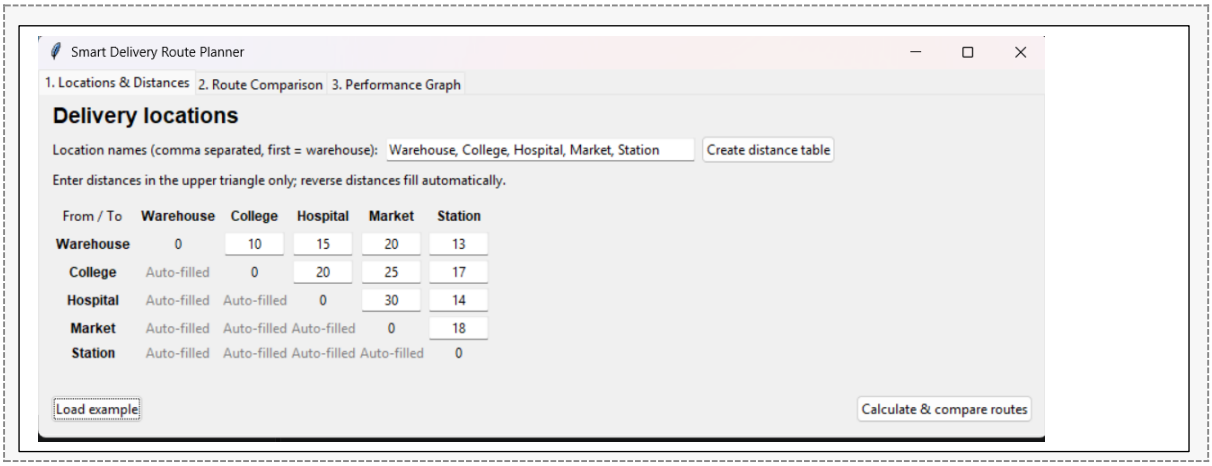


Figure 4. Entering delivery locations and distances (reverse distances auto-fill).

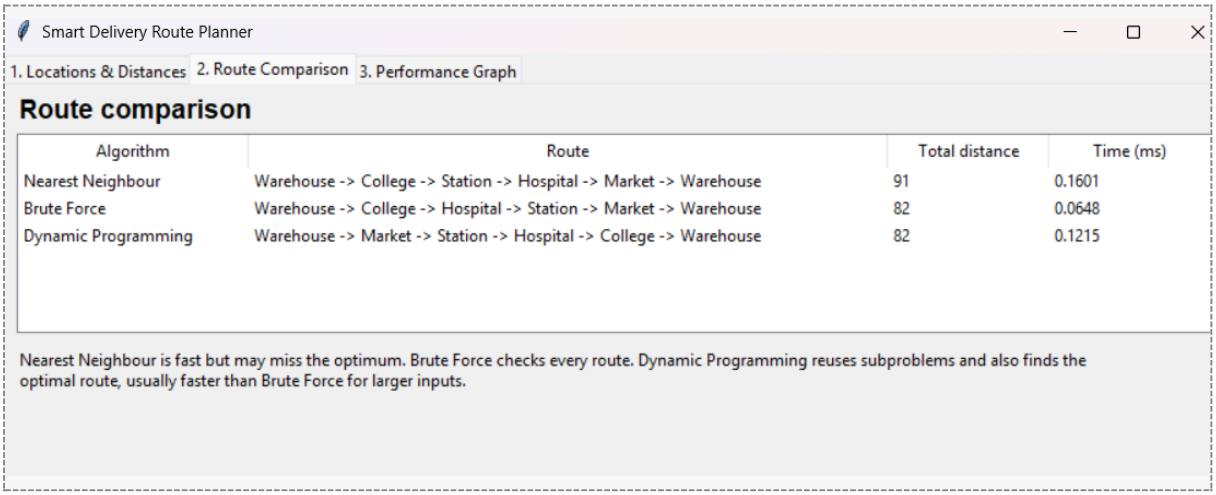


Figure 5. Routes, total distances and times from all three algorithms.

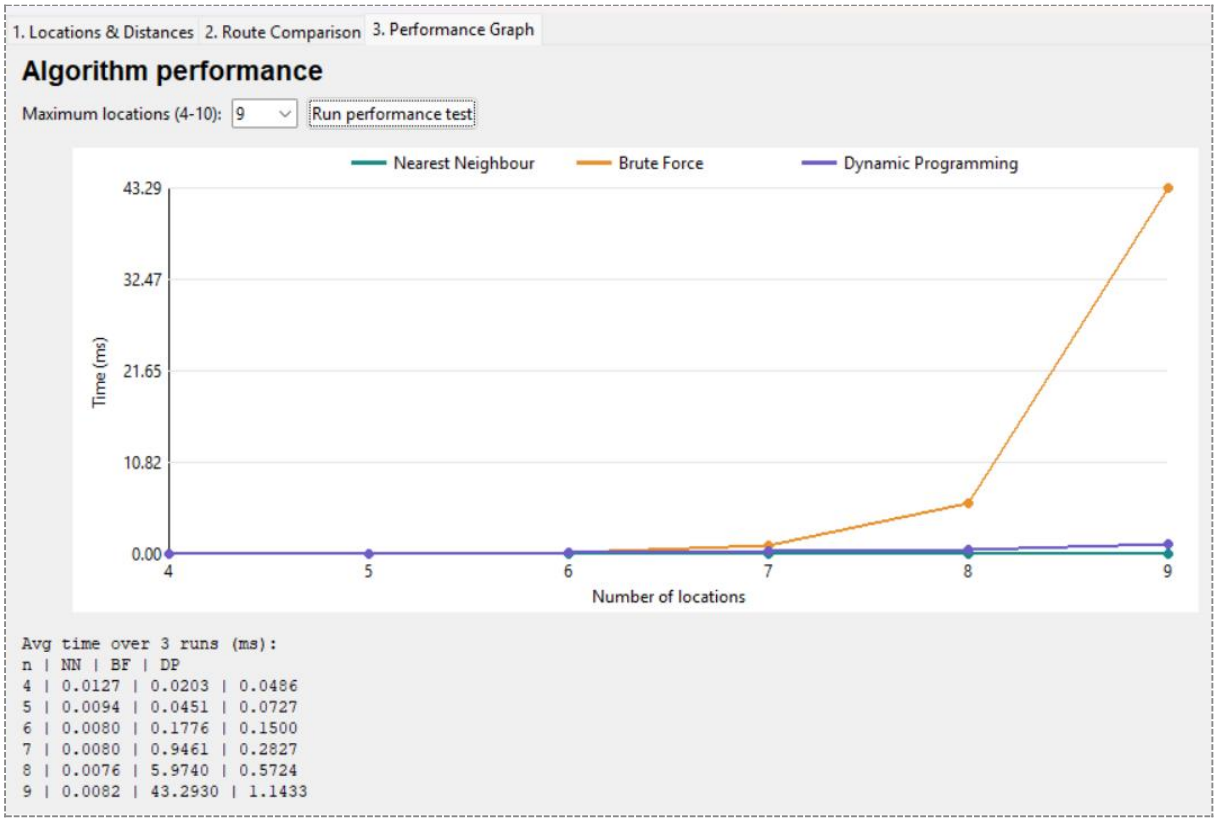


Figure 6. Execution-time comparison for 4 to 9 locations (performance graph and measured table).

7.2 Route Comparison

Table 8 shows the result of running the three algorithms on the same five-location matrix from Table 5. Nearest Neighbour gave a route of 91 units. Brute Force and Dynamic Programming both gave the optimal route of 82 units. Nearest Neighbour was the fastest, but its route was 9 units (about 11 percent) longer than the optimum. At this tiny size, Dynamic Programming is the slowest because it has the overhead of building and storing its tables.

Algorithm	Distance	Optimal?	Time (ms)
Nearest Neighbour	91	Not guaranteed	0.0250
Brute Force	82	Yes	0.0598
Dynamic Programming	82	Yes	0.1821

Table 8. Route comparison for 5 locations on the same input matrix.

7.3 Performance Measurement

To study how the algorithms scale, each one was run on generated symmetric matrices with 4 to 9 locations. For each size, the same matrix was given to all three algorithms, and the time was averaged over three runs. Table 9 and Figure 7 show the measured times.

n	NN (ms)	BF (ms)	DP (ms)
4	0.0069	0.0113	0.0293
5	0.0046	0.0316	0.0637
6	0.0061	0.1349	0.1561
7	0.0080	0.8467	0.4370
8	0.0116	7.4303	1.2579
9	0.0097	37.6347	1.7277

Table 9. Average execution time by input size (n = total locations including the warehouse; NN = Nearest Neighbour, BF = Brute Force, DP = Dynamic Programming).

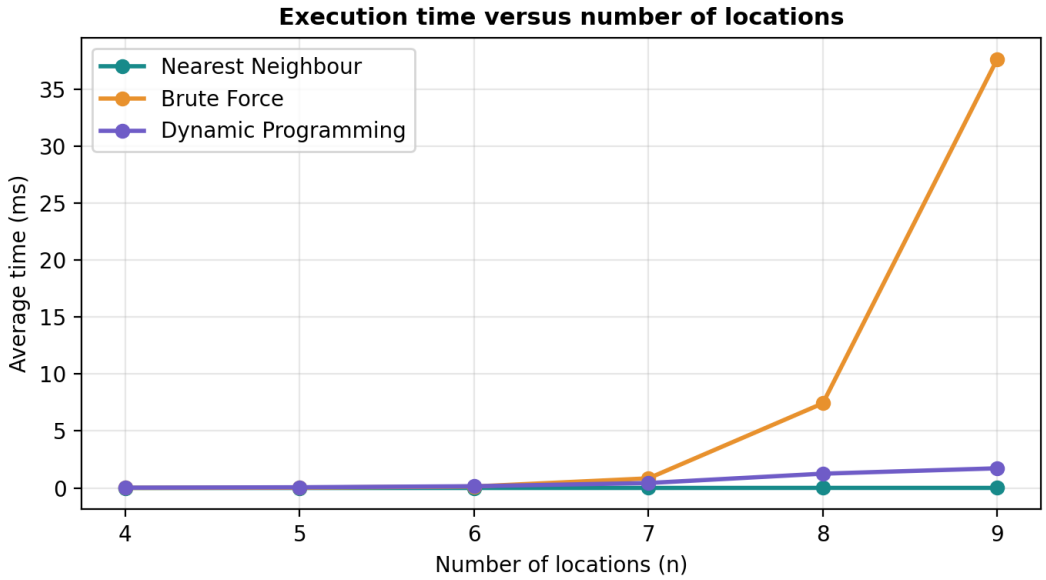


Figure 7. Execution time versus number of locations.

On a linear scale, the Nearest Neighbour and Dynamic Programming lines look nearly flat next to Brute Force. Figure 8 shows the same data on a logarithmic time axis, which makes the differences at small sizes visible.

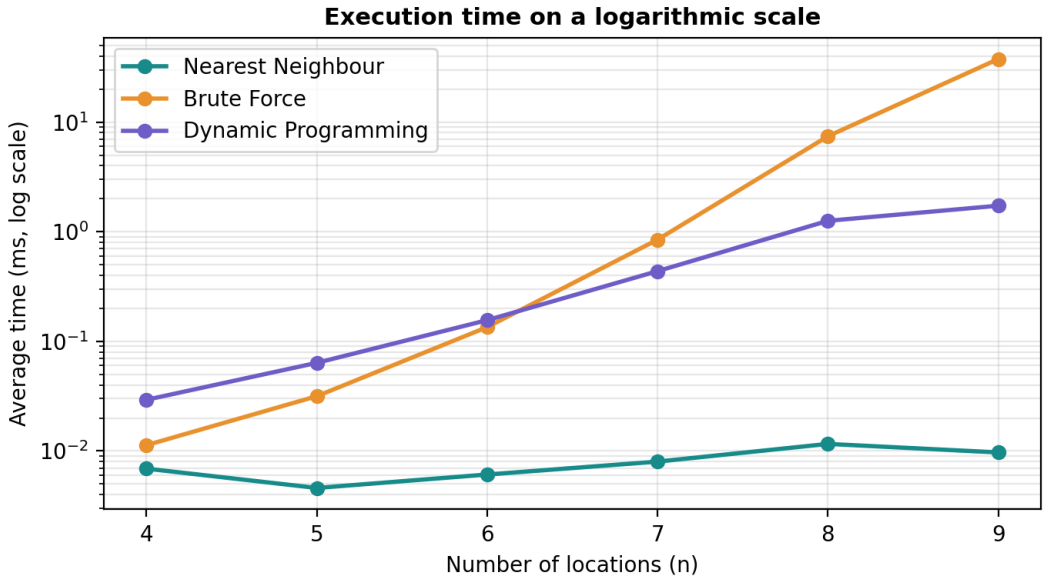


Figure 8. The same measurements on a logarithmic scale.

7.4 Test Cases

The input validation, the route results and the agreement between the algorithms were tested with the cases in Table 10.

No.	Test case	Expected result	Status
1	Load the example and calculate	NN = 91, BF = 82, DP = 82	Pass
2	Enter only one location name	Error: “Enter 2 to 10 different location names.”	Pass
3	Enter more than 10 location names	Same error message	Pass
4	Enter a duplicate location name	Same error message	Pass
5	Leave a distance box empty or type text	Error: “Invalid distance for X - Y.”	Pass
6	Enter a negative distance	Error: “Distances must be non-negative.”	Pass
7	Press Calculate before creating the table	Error: “Create the distance table first.”	Pass
8	Enter a distance in the upper triangle	The reverse distance is used automatically (symmetric matrix)	Pass
9	Two locations only	Route of the form 0 → 1 → 0 with double the distance, same for all algorithms	Pass
10	700 random matrices, n = 2 to 8 (100 for each n)	BF and DP give equal distances; the DP route is a valid tour; NN is never shorter than the optimum	Pass (0 mismatches)
11	Run the performance test for n = 4 to 9	Graph and table are drawn for every size	Pass

Table 10. Test cases and results.

In test case 10, the Nearest Neighbour route was longer than the optimal one in 300 of the 700 matrices, which confirms that the greedy method is only a heuristic. The Brute Force and Dynamic Programming distances matched in all 700 cases, which is evidence that the Held–Karp implementation and its route reconstruction are correct.

7.5 Discussion of Results

- **Quality versus speed:** Nearest Neighbour stayed below 0.012 ms at every size, but its route on the sample was 91 against the optimal 82.
- **Crossover point:** Brute Force is faster than Dynamic Programming for 4 to 6 locations because Dynamic Programming has more overhead per step. From 7 locations onward, Dynamic Programming is faster (0.4370 ms against 0.8467 ms).
- **Growth of Brute Force:** its time grows by about 5 to 9 times for each extra location from 6 to 9 (0.1349, 0.8467, 7.4303, 37.6347 ms). This matches factorial growth, where moving from n to n + 1 multiplies the work by roughly n.

- **Growth of Dynamic Programming:** its time grows by about 3 to 4 times per extra location at the start, then by less (1.2579 to 1.7277 ms from 8 to 9), which fits the 2^n pattern better than n!.
- **Size 9:** Brute Force took 37.63 ms and Dynamic Programming 1.73 ms, so Brute Force was about 22 times slower.

These are real measurements on one computer, so the exact milliseconds will differ on another machine, and for very small times the readings are affected by the clock resolution and by other programs running at the same time. The overall pattern, however, agrees with the theory in Table 4.

CHAPTER – 8 DISCUSSION

8.1 Advantages

- Compares three different algorithm-design techniques on exactly the same input.
- Shows both a heuristic approach and two optimal approaches.
- Includes the return journey to the warehouse in the total distance.
- Provides a graphical interface for input, validation and visualisation.
- Gives both theoretical and experimental analysis.
- Needs no installation beyond Python, because it uses only the standard library.

8.2 Limitations

- Distances have to be entered manually.
- Distances are assumed to be symmetric, so one-way roads cannot be modelled.
- No traffic or real-time road conditions are considered.
- Brute Force becomes impractical as the number of locations grows.
- Dynamic Programming is also exponential in time and memory, which is why the input is limited to 10 locations.
- Timings come from a single machine and a small number of runs.

8.3 Future Scope

- Use a mapping API to obtain real road distances automatically.
- Support one-way roads and asymmetric distances.
- Add traffic and estimated delivery times.
- Support multiple vehicles and delivery constraints such as capacity and time windows.
- Add more scalable heuristics, such as 2-opt improvement or branch and bound, for larger inputs.
- Draw the route on a map-like diagram inside the interface.

CHAPTER – 9 CONCLUSION

This mini project implemented three algorithms for the delivery-route problem and compared a greedy approach with exhaustive search and dynamic programming. The Smart Delivery Route Planning System accepts locations and distances, validates them, runs Nearest Neighbour, Brute Force and Held–Karp on the same matrix, and reports the route, total distance and execution time of each, together with a performance graph.

The results show how algorithm choice affects both efficiency and solution quality. On the sample input, the greedy Nearest Neighbour route was 91 units while the optimal route found by Brute Force and Dynamic Programming was 82 units. In the timing experiment, Brute Force was faster for 4 to 6 locations, but Dynamic Programming overtook it from 7 locations, and at 9 locations it was about 22 times faster (1.73 ms against 37.63 ms). These measurements match the theoretical complexities of $O(n^2)$, $O(n!)$ and $O(n^2 \cdot 2^n)$.

The main lesson is that the best algorithm depends on the balance between execution time, memory usage and the need for an optimal route. A greedy method is suitable when speed matters most and a good-enough route is acceptable. Dynamic Programming is the better choice when the optimal route is required and the number of locations is moderate. Brute Force is useful mainly as a simple baseline for very small inputs. The project also shows that even a basic problem from algorithm theory can be turned into a clear and usable application.

REFERENCES

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, MIT Press: algorithm analysis and dynamic programming.

[2] M. Held and R. M. Karp, “A Dynamic Programming Approach to Sequencing Problems,” Journal of the Society for Industrial and Applied Mathematics, vol. 10, no. 1, 1962.

[3] Python Documentation: itertools.permutations and itertools.combinations, docs.python.org.

[4] Python Documentation: time module (time.perf_counter), docs.python.org.

[5] Python Documentation: tkinter, Python interface to Tcl/Tk, docs.python.org.

[6] DAA Mini Project Rules and Guidelines; DAA Mini Project Presentation Format (course documents).

APPENDIX A – SOURCE CODE

A.1 route_algorithms.py

```
import itertools
from itertools import combinations


INF = float("inf")



def calculate_route_distance(route, dist):
    """Total distance of a route, including the return to the warehouse."""
    total = 0
    for i in range(len(route) - 1):
        total += dist[route[i]][route[i + 1]]
    total += dist[route[-1]][route[0]]  # return trip
    return total



def nearest_neighbour(dist):
    """Greedy: always go to the nearest unvisited location."""
    n = len(dist)
    route = [0]
    visited = [False] * n
    visited[0] = True
    current = 0
    while len(route) < n:
        nxt = None
        for j in range(n):
            if not visited[j] and (nxt is None or dist[current][j] < dist[current][nxt]):
                nxt = j
        route.append(nxt)
        visited[nxt] = True
        current = nxt
    return route, calculate_route_distance(route, dist)



def brute_force(dist):
    """Try every order of the other locations, keep the minimum."""
    n = len(dist)
    best_route, best = None, INF
    for perm in itertools.permutations(range(1, n)):
        route = [0] + list(perm)
        d = calculate_route_distance(route, dist)
        if d < best:
            best, best_route = d, route
    return best_route, best



def dynamic_programming(dist):
    """Held-Karp: dp[(mask, j)] = shortest path starting at 0, visiting
    the locations in mask, and ending at j. Bit (k-1) of mask = location k."""
    n = len(dist)
    if n == 1:
        return [0], 0
    dp, parent = {}, {}
    for j in range(1, n):
        dp[(1 << (j - 1), j)] = dist[0][j]
        parent[(1 << (j - 1), j)] = 0


    for size in range(2, n):
        for subset in combinations(range(1, n), size):
            mask = sum(1 << (k - 1) for k in subset)
            for j in subset:
                prev = mask ^ (1 << (j - 1))
                best, best_k = INF, None
                for k in subset:
                    if k == j:
                        continue
                    cost = dp[(prev, k)] + dist[k][j]
                    if cost < best:
                        best, best_k = cost, k
                dp[(mask, j)] = best
                parent[(mask, j)] = best_k
```

```
    full = (1 << (n - 1)) - 1
    best, last = INF, None
    for j in range(1, n):
        cost = dp[(full, j)] + dist[j][0]
        if cost < best:
            best, last = cost, j

    # rebuild route using the parent map
    route, mask, j = [], full, last
    while j != 0:
        route.append(j)
        pj = parent[(mask, j)]
        mask ^= 1 << (j - 1)
        j = pj
    route.append(0)
    route.reverse()
    return route, best
```

A.2 smart_delivery_gui.py

```
import random
import time
import tkinter as tk
from tkinter import ttk, messagebox

from route_algorithms import nearest_neighbour, brute_force, dynamic_programming

ALGOS = [("Nearest Neighbour", nearest_neighbour, "#178a8a"),
         ("Brute Force", brute_force, "#e8912d"),
         ("Dynamic Programming", dynamic_programming, "#6f5cc7")]



def timed(func, dist):
    start = time.perf_counter()
    route, d = func(dist)
    return route, d, (time.perf_counter() - start) * 1000  # ms



class App:
    def __init__(self, root):
        root.title("Smart Delivery Route Planner")
        root.geometry("900x620")
        self.names, self.entries = [], {}


        nb = ttk.Notebook(root)
        nb.pack(fill="both", expand=True)
        self.nb = nb
        self.t1, self.t2, self.t3 = ttk.Frame(nb), ttk.Frame(nb), ttk.Frame(nb)
        nb.add(self.t1, text="1. Locations & Distances")
        nb.add(self.t2, text="2. Route Comparison")
        nb.add(self.t3, text="3. Performance Graph")
        self.build_tab1()
        self.build_tab2()
        self.build_tab3()


    # ---------- Tab 1: input ----------
    def build_tab1(self):
        ttk.Label(self.t1, text="Delivery locations", font=("Arial", 14,
"bold")).pack(anchor="w", padx=10, pady=5)
        top = ttk.Frame(self.t1)
        top.pack(fill="x", padx=10)
        ttk.Label(top, text="Location names (comma separated, first =
warehouse):").pack(side="left")
        self.name_var = tk.StringVar(value="Warehouse, College, Hospital, Market, Station")
        ttk.Entry(top, textvariable=self.name_var, width=45).pack(side="left", padx=5)
        ttk.Button(top, text="Create distance table",
command=self.create_table).pack(side="left")


        ttk.Label(self.t1, text="Enter distances in the upper triangle only; reverse distances
fill automatically.").pack(anchor="w", padx=10, pady=5)
        self.grid_frame = ttk.Frame(self.t1)
        self.grid_frame.pack(padx=10, pady=5, anchor="w")


        bottom = ttk.Frame(self.t1)
        bottom.pack(side="bottom", fill="x", padx=10, pady=10)
```

```
        ttk.Button(bottom, text="Load example", command=self.load_example).pack(side="left")
        ttk.Button(bottom, text="Calculate & compare routes",
command=self.calculate).pack(side="right")


    def create_table(self):
        names = [x.strip() for x in self.name_var.get().split(",") if x.strip()]
        if not 2 <= len(names) <= 10 or len(set(names)) != len(names):
            messagebox.showerror("Error", "Enter 2 to 10 different location names.")
            return False
        self.names = names
        for w in self.grid_frame.winfo_children():
            w.destroy()
        self.entries = {}
        n = len(names)
        ttk.Label(self.grid_frame, text="From / To").grid(row=0, column=0, padx=3, pady=3)
        for i, nm in enumerate(names):
            ttk.Label(self.grid_frame, text=nm, font=("Arial", 9, "bold")).grid(row=0, column=i +
1, padx=3)
            ttk.Label(self.grid_frame, text=nm, font=("Arial", 9, "bold")).grid(row=i + 1,
column=0, padx=3)
        for i in range(n):
            for j in range(n):
                if i == j:
                    ttk.Label(self.grid_frame, text="0").grid(row=i + 1, column=j + 1)
                elif j > i:
                    e = ttk.Entry(self.grid_frame, width=8, justify="center")
                    e.grid(row=i + 1, column=j + 1, padx=2, pady=2)
                    self.entries[(i, j)] = e
                else:
                    ttk.Label(self.grid_frame, text="Auto-filled", foreground="gray").grid(row=i
+ 1, column=j + 1)
        return True


    def load_example(self):
        self.name_var.set("Warehouse, College, Hospital, Market, Station")
        self.create_table()
        data = {(0, 1): 10, (0, 2): 15, (0, 3): 20, (0, 4): 13,
                (1, 2): 20, (1, 3): 25, (1, 4): 17,
                (2, 3): 30, (2, 4): 14, (3, 4): 18}
        for key, val in data.items():
            self.entries[key].insert(0, str(val))


    def get_matrix(self):
        n = len(self.names)
        if n == 0:
            raise ValueError("Create the distance table first.")
        m = [[0] * n for _ in range(n)]
        for (i, j), e in self.entries.items():
            try:
                v = float(e.get())
            except ValueError:
                raise ValueError(f"Invalid distance for {self.names[i]} - {self.names[j]}.")
            if v < 0:
                raise ValueError("Distances must be non-negative.")
            m[i][j] = m[j][i] = v  # symmetric mirror
        return m


    def calculate(self):
        try:
            dist = self.get_matrix()
        except ValueError as err:
            messagebox.showerror("Input error", str(err))
            return
        self.tree.delete(*self.tree.get_children())
        for name, func, _ in ALGOS:
            route, d, t = timed(func, dist)
            path = " -> ".join(self.names[i] for i in route + [0])
            self.tree.insert("", "end", values=(name, path, f"{d:g}", f"{t:.4f}"))
        self.nb.select(self.t2)


    # ---------- Tab 2: results ----------
    def build_tab2(self):
        ttk.Label(self.t2, text="Route comparison", font=("Arial", 14, "bold")).pack(anchor="w",
padx=10, pady=5)
        cols = ("Algorithm", "Route", "Total distance", "Time (ms)")
        self.tree = ttk.Treeview(self.t2, columns=cols, show="headings", height=6)
        for c, w in zip(cols, (150, 450, 100, 100)):
            self.tree.heading(c, text=c)
            self.tree.column(c, width=w)
        self.tree.pack(fill="x", padx=10)
        ttk.Label(self.t2, wraplength=800, text=(
```

```
            "Nearest Neighbour is fast but may miss the optimum. Brute Force checks every route.
"
            "Dynamic Programming reuses subproblems and also finds the optimal route, "
            "usually faster than Brute Force for larger inputs.")).pack(anchor="w", padx=10,
pady=10)

    # ---------- Tab 3: benchmark ----------
    def build_tab3(self):
        ttk.Label(self.t3, text="Algorithm performance", font=("Arial", 14,
"bold")).pack(anchor="w", padx=10, pady=5)
        top = ttk.Frame(self.t3)
        top.pack(fill="x", padx=10)
        ttk.Label(top, text="Maximum locations (4-10):").pack(side="left")
        self.max_n = ttk.Combobox(top, values=list(range(4, 11)), width=4, state="readonly")
        self.max_n.set(9)
        self.max_n.pack(side="left", padx=5)
        ttk.Button(top, text="Run performance test", command=self.benchmark).pack(side="left")
        self.canvas = tk.Canvas(self.t3, width=800, height=330, bg="white")
        self.canvas.pack(padx=10, pady=10)
        self.table_lbl = ttk.Label(self.t3, justify="left", font=("Courier", 9))
        self.table_lbl.pack(anchor="w", padx=10)


    def benchmark(self):
        top_n = int(self.max_n.get())
        sizes = list(range(4, top_n + 1))
        results = {name: [] for name, _, _ in ALGOS}
        for n in sizes:
            for name, func, _ in ALGOS:
                total = 0
                for _ in range(3):  # average of 3 runs
                    random.seed(n)
                    m = [[0] * n for _ in range(n)]
                    for i in range(n):
                        for j in range(i + 1, n):
                            m[i][j] = m[j][i] = random.randint(5, 50)
                    total += timed(func, m)[2]
                results[name].append(total / 3)
            self.t3.update()
        self.draw_graph(sizes, results)
        lines = ["Avg time over 3 runs (ms):", "n | NN | BF | DP"]
        for k, n in enumerate(sizes):
            lines.append(f"{n} | " + " | ".join(f"{results[a[0]][k]:.4f}" for a in ALGOS))
        self.table_lbl.config(text="\n".join(lines))


    def draw_graph(self, sizes, results):
        c = self.canvas
        c.delete("all")
        L, R, T, B = 70, 780, 30, 290
        ymax = max(max(v) for v in results.values()) or 1
        c.create_line(L, T, L, B)
        c.create_line(L, B, R, B)
        c.create_text(20, (T + B) // 2, text="Time (ms)", angle=90)
        c.create_text((L + R) // 2, 320, text="Number of locations")
        for k in range(5):
            y = B - (B - T) * k / 4
            c.create_line(L, y, R, y, fill="#e5e5e5")
            c.create_text(L - 5, y, text=f"{ymax * k / 4:.2f}", anchor="e")
        step = (R - L) / (len(sizes) - 1)
        for k, n in enumerate(sizes):
            c.create_text(L + k * step, B + 12, text=str(n))
        for lx, (name, _, colour) in zip((200, 360, 520), ALGOS):
            c.create_line(lx, 12, lx + 25, 12, fill=colour, width=3)
            c.create_text(lx + 30, 12, text=name, anchor="w")
            pts = [(L + k * step, B - (B - T) * v / ymax) for k, v in enumerate(results[name])]
            for (x1, y1), (x2, y2) in zip(pts, pts[1:]):
                c.create_line(x1, y1, x2, y2, fill=colour, width=2)
            for x, y in pts:
                c.create_oval(x - 3, y - 3, x + 3, y + 3, fill=colour, outline=colour)



if __name__ == "__main__":
    root = tk.Tk()
    App(root)
    root.mainloop()
```